



UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
CAMPUS SÃO CRISTOVÃO

Disciplina: Inteligência Artificial
Docente: Hendrik Teixeira Macedo

Relatório Técnico - Árvores de Decisão

Vitor Araújo Andrade
Alex Lemos de Souza Pinheiro
Mateus da Silva Barreto

São Cristóvão, SE.

Fevereiro de 2026

Introdução

Objetivo: Utilizar árvores de decisão para identificar números de 0 a 9 com mais de 90% de precisão e utilizando o pseudo-código fornecido abaixo:

```
function LEARN-DECISION-TREE(examples, attributes, parent_examples) returns a tree

if examples is empty then return PLURALITY-VALUE(parent_examples)
else if all examples have the same classification then return the classification
else if attributes is empty then return PLURALITY-VALUE(examples)
else
     $A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$ 
    tree  $\leftarrow$  a new decision tree with root test A
    for each value v of A do
        exs  $\leftarrow \{e : e \in \text{examples} \text{ and } e.A = v\}$ 
        subtree  $\leftarrow$  LEARN-DECISION-TREE(exs, attributes – A, examples)
        add a branch to tree with label (A = v) and subtree subtree
    return tree
```

OBS: A função IMPORTANCE é descrita na Seção 19.3.3 do livro. Ela está relacionada ao cálculo do Ganho de Informação (INFORMATION-GAIN) e da Entropia (ENTROPY). A função PLURALITY-VALUE seleciona o valor de saída mais comum entre um conjunto de exemplos; os empates são decididos aleatoriamente.

Fundamentação Teórica

O pseudocódigo funciona através de uma estratégia recursiva: ele verifica se os dados são puros ou se os atributos acabaram para criar nós finais (folhas). Caso contrário, ele seleciona o atributo mais importante para dividir o conjunto de dados em ramos baseados em seus valores. Para cada ramo, o processo se repete criando subárvores até que todo o caminho de decisão seja mapeado.

Implementação das Funções do ID3

Para a construção do algoritmo, as funções foram implementadas em Python utilizando dicionários para representar a estrutura da árvore.

Funções de Utilidade e Acesso

Estas funções gerenciam como o algoritmo "enxerga" os dados (imagens de dígitos).

- `get_classification(example)`: Retorna a classe real (target).
- `get_value(example, attribute)`: Busca o valor de um pixel específico.
- `plurality_value(examples)`: Define o veredito por maioria em caso de empate ou parada.

```
def get_classification(example):
    """Extraí o valor do alvo (target/label) de um exemplo."""
    return example['target']

def get_value(example, attribute):
    """Busca o valor de um atributo (ex: se o px_34 é 0 ou 1)."""
    return example[attribute]

def plurality_value(examples):
    """
    VOTAÇÃO DE MAIORIA: Se o algoritmo precisa parar, ele escolhe o número
    que mais aparece naquele conjunto. Se empatar, pega o primeiro.
    """
    if not examples:
        return None
    labels = [get_classification(e) for e in examples]
    contagem = Counter(labels)
    return contagem.most_common(1)[0][0]
```

Funções Matemáticas

Aqui reside a lógica de seleção de atributos baseada na Entropia de Shannon.

- `entropy(examples)`: Mede a desordem dos dados. Quanto mais misturadas as classes, maior a entropia.
- `importance(attribute, examples)`: Mede o Ganho de Informação (o quanto a entropia diminui ao dividirmos os dados por este atributo).

```
def entropy(examples):  
    """  
    ENTROPIA DE SHANNON: Mede a desordem ou incerteza.  
    Fórmula:  $H(S) = - \sum p_i * \log_2(p_i)$   
    Se todos forem do mesmo número, a entropia é 0 (ordem total).  
    """  
    if not examples:  
        return 0.0  
    labels = [get_classification(e) for e in examples]  
    total_examples = len(labels)  
    contagem_classes = Counter(labels)  
  
    ent = 0.0  
    for contagem in contagem_classes.values():  
        probabilidade = contagem / total_examples  
        # O log2 mede a 'surpresa' da informação  
        ent -= probabilidade * math.log2(probabilidade)  
    return ent
```

```

def importance(attribute, examples):
    """
    GANHO DE INFORMAÇÃO: É a 'utilidade' de um atributo.
    Calcula quanto a entropia diminui se dividirmos os dados por este atributo.
    Atributos que separam bem os números ganham notas maiores.
    """
    if not examples:
        return 0.0
    entropia_base = entropy(examples)
    total_examples = len(examples)

    # Agrupa exemplos pelos valores do atributo (0 ou 1)
    subconjuntos = {}
    for e in examples:
        v = get_value(e, attribute)
        if v not in subconjuntos:
            subconjuntos[v] = []
        subconjuntos[v].append(e)

    # Calcula a média ponderada da entropia dos subconjuntos
    entropia_esperada = 0.0
    for subconjunto in subconjuntos.values():
        peso = len(subconjunto) / total_examples
        entropia_esperada += peso * entropy(subconjunto)

    # O ganho é a diferença entre a dúvida inicial e a dúvida após o teste
    return entropia_base - entropia_esperada

```

Funções de Controle e Fluxo

Estas funções determinam quando a divisão de dados deve parar ou quais caminhos seguir.

- `all_same_classification(examples)`: Esta função verifica a pureza de um nó. Se todos os exemplos no conjunto atual pertencem à mesma classe, não há necessidade de continuar a divisão. É um dos principais critérios de parada do ID3.
- `possible_values(attribute, examples)`: Mapeia todos os valores distintos que um determinado atributo assume no conjunto de dados. Em nosso caso de imagens binarizadas, os valores possíveis são geralmente [0, 1], representando a ausência ou presença de tinta em um pixel.

```
def all_same_classification(examples):
    """
    PUREZA: Verifica se todos os exemplos são do mesmo número (ex: tudo '8').
    Se sim, a árvore não precisa mais ser dividida aqui.
    """
    if not examples:
        return True
    primeira_classe = get_classification(examples[0])
    for e in examples:
        if get_classification(e) != primeira_classe:
            return False
    return True

def possible_values(attribute, examples):
    """Identifica quais caminhos o atributo pode seguir (geralmente 0 ou 1)."""
    return list(set(get_value(e, attribute) for e in examples))

# --- FUNÇÕES MATEMÁTICAS (CÉREBRO) ---
```

Função Principal

Esta é a função recursiva que constrói a árvore seguindo o pseudocódigo. Dentro dela são realizados esses passos:

- Casos Base (Poda e Parada): Verifica se todos os exemplos são da mesma classe, se os atributos acabaram ou se os limites de profundidade (max_depth) e quantidade mínima de amostras (5) foram atingidos.
- Seleção do Atributo: Escolhe o melhor atributo através da função importance.
- Ramificação: Para cada valor possível do melhor atributo, ela cria um novo subconjunto de dados e chama a si mesma novamente para construir as subárvores.

```

def learn_decision_tree(examples, attributes, parent_examples, max_depth=10, depth=0):
    """
    CONSTRUÇÃO DA ÁRVORE: Implementação do pseudocódigo do livro de Russell & Norvig.
    Cria uma estrutura de dicionários aninhados que representa a árvore.
    """
    # CASO BASE 1: Se não há mais exemplos, usa a maioria do pai (evita galhos vazios)
    if not examples:
        return plurality_value(parent_examples)

    # CASO BASE 2: Pré-poda (Pre-pruning). Se o grupo é muito pequeno, para aqui.
    if len(examples) < 5:
        return plurality_value(examples)

    # CASO BASE 3: Se todos os exemplos são da mesma classe, virou uma folha.
    elif all_same_classification(examples):
        return get_classification(examples[0])

    # CASO BASE 4: Se não há mais atributos para testar ou atingiu o limite de altura.
    elif not attributes or depth >= max_depth:
        return plurality_value(examples)

    else:
        # ESCOLHA DO LÍDER: Seleciona o atributo com o maior Ganho de Informação
        A = max(attributes, key=lambda a: importance(a, examples))

        # Cria o nó da árvore (um dicionário onde a chave é o nome do atributo)
        tree = {A: {}}

        # Para cada valor possível deste atributo (ex: 0 e 1), cria um sub-galho
        for v in possible_values(A, examples):
            exs = [e for e in examples if get_value(e, A) == v]
            # Remove o atributo já usado para não repetir o teste no próximo nível
            remaining_attributes = [attr for attr in attributes if attr != A]

            # RECURSIVIDADE: Chama a função para construir o próximo nível
            subtree = learn_decision_tree(exs, remaining_attributes, examples, max_depth, depth + 1)
            tree[A][v] = subtree

        return tree

```

Implementação Técnica e Mapeamento Teórico

Modelagem de dados

example - representa uma única unidade de dados do conjunto, neste caso, a imagem de um dígito. Em python, implementado como dicionário, onde

- *chaves: atributos ou características extraídas*
- *valores: estados dos atributos*
- *target - chave especial - classificação real do exemplo*

Comparação ativa entre pseudocódigo e implementação em Python

Pseudocódigo (Lógica Teórica)	Implementação em Python
function LEARN-DECISION-TREE(ex amples, attributes, parent_examples)	def learn_decision_tree(examples, attributes, parent_examples, ...):
if examples is empty then return PLURALITY-VALUE(parent_ examples)	if not examples: return plurality_value(parent_exampl es)
else if all examples have the same classification then return the classification	elif all_same_classification(exampl es): return get_classification(examples[0])

else if attributes is empty then return PLURALITY-VALUE(examples)	elif not attributes or depth >= max_depth: return plurality_value(examples)
$A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$	$A = \max(\text{attributes}, \text{key}=\lambda a: \text{importance}(a, \text{examples}))$
tree $\leftarrow$ a new decision tree with root test A	tree = {A: {}}
for each value v of A do	for v in possible_values(A, examples):
$\text{exs} \leftarrow \{e : e \in \text{examples} \text{ and } e.A = v\}$	$\text{exs} = [e \text{ for } e \text{ in examples if } \text{get_value}(e, A) == v]$
subtree $\leftarrow$ LEARN-DECISION-TREE(exs, attributes - A, examples)	subtree = learn_decision_tree(exs, remaining_attributes, examples, ...)
add a branch to tree with label (A=v) and subtree subtree	tree[A][v] = subtree

return tree	return tree
-------------	-------------

Ao utilizar somente este arquivo do ID3 para testes, notamos inicialmente uma precisão baixa, **37,78%**.

Ao melhorar a lógica de avaliação conseguimos chegar a cerca de 87%.

Daí decidimos melhorar a abordagem e torná-la mais “democrática”, utilizando uma implementação nossa do Random_Forest.

Implementação do Random_Forest_Caseiro

A **Random Forest** funciona criando múltiplas árvores de decisão independentes, treinadas com subconjuntos aleatórios de dados e atributos para garantir a diversidade. Cada árvore gera uma previsão individual percorrendo seus nós até encontrar uma classificação. No final, o algoritmo utiliza um **sistema de votação majoritária** para combinar os resultados de todas as árvores. Essa abordagem coletiva reduz erros individuais e aumenta a estabilidade da acurácia final do modelo.

Implementação

```
treinar_random_forest(dados_treino, atributos,  
n_arvores=15, profundidade_maxima=10)
```

Funcionamento:

- **Criação da Floresta:** Ela cria uma lista vazia chamada floresta que armazena as N árvores de decisão independentes
- **Bagging:** Atribui para cada árvore, uma amostra diferente dos dados de treino
- **Seleção Aleatória de Atributos:** Em vez de usar todos os 68 atributos disponíveis (pixels + formas) em cada nó, a função sorteia um subconjunto aleatório (limite de 25) usando `random.sample`, o que garante que as árvores não fiquem todas iguais e explorem caminhos de decisão variados.
- **Integração com o ID3:** Para cada amostra e conjunto de atributos sorteados, a função chama `learn_decision_tree`, repassando o parâmetro `profundidade_maxima` para controlar o crescimento de cada árvore individualmente.
- **Consolidação:** Após o loop, todas as árvores prontas são armazenadas na lista final, que será utilizada posteriormente para realizar a votação majoritária nas previsões.

```
def treinar_random_forest(dados_treino, atributos, n_arvores=15, profundidade_maxima=10):
    floresta = []
    print(f"Construindo Floresta com {n_arvores} árvores (Profundidade: {profundidade_maxima})...")

    for i in range(n_arvores):
        # TÉCNICA 1: BAGGING (Bootstrap Aggregating)
        # Sorteia aleatoriamente 70% dos dados para treinar esta árvore específica.
        # Isso faz com que cada árvore veja dados ligeiramente diferentes.
        amostra = random.choices(dados_treino, k=int(len(dados_treino) * 0.7))

        # TÉCNICA 2: FEATURE SELECTION (Subspace Method)
        # Em vez de olhar para os 64 pixels + 4 formas, sorteia no máximo 25 atributos.
        # Isso impede que o pixel mais "forte" domine todas as árvores, forçando a IA a achar outros padrões.
        attrs_sorteados = random.sample(atributos, k=min(25, len(atributos)))

        # Treina uma árvore do zero com esses dados e atributos restritos
        arvore = learn_decision_tree(amostra, attrs_sorteados, [], max_depth=profundidade_maxima)

        floresta.append(arvore)
        print(f" > Árvore {i+1} pronta.")

    return floresta
```

Arquivo de Testes

Para testar as implementações discutidas anteriormente, criamos um arquivo de teste que mostra principalmente a acurácia em cada número e uma média geral

(A implementação mostrada se refere a versão final, direcionada a utilização da Random Forest)

Funções:

- extrair_todas_as_formas(pixels_64)
- prever_individual(no, ex)
- executar()

Funcionamento:

extrair_todas_as_formas(pixels_64)

Esta função é a camada de **Engenharia de Atributos**. Ela não olha apenas para pixels isolados, mas tenta obter padrões geométricos como um ser humano faria, identificando linhas, curvas, centros preenchidos e cruzamentos

```
def extrair_todas_as_formas(pixels_64):  
    """  
    ENGENHARIA DE ATRIBUTOS:  
    Transforma o vetor de 64 pixels em uma matriz 8x8 e procura padrões geométricos.  
    Isso ajuda a árvore de decisão a entender conceitos como "curva" ou "reta".  
    """  
    matriz = pixels_64.reshape(8, 8)  
    caracs = {}  
  
    # 1. Reta no Topo: Verifica se existem 3 pixels seguidos na linha superior  
    # Útil para identificar o topo do número '7' ou '5'.  
    reta = 0  
    for c in range(6):  
        if matriz[0, c] == 1 and matriz[0, c+1] == 1 and matriz[0, c+2] == 1:  
            reta = 1; break  
    caracs['f_reta_topo'] = reta  
  
    # 2. Curva na Base: Soma pixels na parte inferior central  
    # Útil para identificar a base arredondada do '0', '6' ou '8'.  
    curva = 0  
    if np.sum(matriz[6:, 2:6]) > 2: curva = 1  
    caracs['f_curva_base'] = curva  
  
    # 3. Cruzamentos: Procura pixels que têm vizinhos em várias direções (como um +)  
    # Útil para identificar o centro de um '8' ou o meio de um '4'.  
    cruz = 0  
    for r in range(1, 7):  
        for c in range(1, 7):  
            if matriz[r,c] == 1 and (matriz[r-1,c]+matriz[r+1,c]+matriz[r,c-1]+matriz[r,c+1]) >= 3:  
                cruz += 1  
    caracs['f_muitos_cruzamentos'] = 1 if cruz > 2 else 0  
  
    # 4. Centro Preenchido: Verifica o miolo da imagem (matriz 2x2 central)  
    # Números como '0' e '8' tendem a ter o centro vazio, enquanto '1' ou '4' podem ter preenchimento.  
    caracs['f_centro_preenchido'] = 1 if np.sum(matriz[3:5, 3:5]) >= 2 else 0  
  
    return caracs
```

prever_individual(no, ex).

É o código que coloca a inteligência em prática,

transformando o conhecimento guardado na árvore em uma resposta concreta.

```
def prever_individual(no, ex):  
    """  
    INFERÊNCIA (Navegação na Árvore):  
    Função recursiva que percorre o dicionário da árvore até chegar em um valor (folha).  
    """  
    # Se o 'no' não for um dicionário, chegamos na folha (o número previsto)  
    if not isinstance(no, dict): return no  
  
    # Pega o atributo que esta parte da árvore quer testar (ex: 'px_34' ou 'f_reta_topo')  
    atrib = list(no.keys())[0]  
    valor = ex.get(atrib)  
  
    # Se o valor do teste não existir no caminho da árvore, tenta um fallback  
    if valor not in no[atrib]:  
        opcoes = [v for v in no[atrib].values() if not isinstance(v, dict)]  
        return opcoes[0] if opcoes else None  
  
    # Continua descendo na árvore  
    return prever_individual(no[atrib][valor], ex)
```

executar()

Ela define o fluxo de trabalho do início ao fim.

1. **Carga e Binarização:** Carrega os dígitos e converte tons de cinza em preto e branco.
2. **Preparação do Dataset:** Converte cada imagem em um dicionário contendo os 64 pixels + as 4 formas extraídas anteriormente.
3. **Divisão de Dados:** Separa **80% para treino** (ensinar a IA) e **20% para teste** (avaliar o desempenho).
4. **Treinamento:** Chama a lógica da Random Forest para criar N árvores independentes, pode ser definido pelo usuário.

5. **Votação:** Para cada exemplo de teste, coleta as previsões de todas as N árvores e usa o Counter para definir o vencedor por **maioria simples**.
6. **Métricas:** Exibe a acurácia final e o relatório detalhado de erros e acertos.

```
def executar():
    """
    PIPELINE PRINCIPAL:
    Carrega dados -> Processa -> Treina -> Avalia.
    """
    # 1. Carregamento e Binarização (Preto e Branco)
    digits = load_digits()
    X_bin = (digits.data > 8).astype(int)
    y = digits.target

    # 2. Preparação do Dataset: Unindo pixels brutos + formas extraídas
    examples = []
    for i in range(len(X_bin)):
        ex = {'px_{j}': X_bin[i][j] for j in range(64)}
        ex.update(extrair_todas_as_formas(X_bin[i]))
        ex['target'] = y[i]
        examples.append(ex)

    # Lista de todos os "campos" que a IA pode analisar
    attrs = [f'px_{i}' for i in range(64)] + ['f_reta_topo', 'f_curva_base', 'f_muitos_cruzamentos', 'f_centro_preenchido']

    # 3. Divisão Treino/Teste
    train, test = train_test_split(examples, test_size=0.2, random_state=42)

    # 4. Treinamento da Floresta (Criando 100 árvores diferentes)
    floresta = treinar_random_forest(train, attrs, n_arvores=100, profundidade_maxima = 12)

    # 5. Avaliação: Cada árvore vota, e a maioria vence
    y_true, y_pred = [], []
    for ex in test:
        votos = [prever_individual(arv, ex) for arv in floresta]
        votos = [v for v in votos if v is not None]
        y_true.append(ex['target'])

        # O Counter pega o elemento mais frequente na lista de votos
        if votos:
            vencedor = Counter(votos).most_common(1)[0][0]
            y_pred.append(vencedor)
        else:
            y_pred.append(0)

    # 6. Relatórios de Performance
    print(f"\nAcurácia da Floresta: {accuracy_score(y_true, y_pred)*100:.2f}%")
    print(classification_report(y_true, y_pred))

# Garante que o treino só comece se o arquivo for executado diretamente
if __name__ == "__main__":
    executar()
```

Com estas funcionalidades é possível testar a nossa implementação da Random Forest, onde conseguimos excelentes resultados como estes:

Construindo Floresta com 20 árvores...

- > Árvore 1 pronta.
- > Árvore 2 pronta.
- > Árvore 3 pronta.
- > Árvore 4 pronta.
- > Árvore 5 pronta.
- > Árvore 6 pronta.
- > Árvore 7 pronta.
- > Árvore 8 pronta.
- > Árvore 9 pronta.
- > Árvore 10 pronta.
- > Árvore 11 pronta.
- > Árvore 12 pronta.
- > Árvore 13 pronta.
- > Árvore 14 pronta.
- > Árvore 15 pronta.
- > Árvore 16 pronta.
- > Árvore 17 pronta.

> Árvore 18 pronta.

> Árvore 19 pronta.

> Árvore 20 pronta.

Acurácia da Floresta: 92.22%

	precision	recall	f1-score	support
0	0.97	0.97	0.97	33
1	0.76	0.93	0.84	28
2	0.92	1.00	0.96	33
3	0.94	0.88	0.91	34
4	1.00	0.96	0.98	46
5	0.94	0.96	0.95	47
6	0.94	0.94	0.94	35
7	0.94	0.97	0.96	34
8	0.89	0.80	0.84	30
9	0.89	0.80	0.84	40
accuracy			0.92	360
macro avg	0.92	0.92	0.92	360
weighted avg	0.92	0.92	0.92	360

> Árvore 98 pronta.

> Árvore 99 pronta.

> Árvore 100 pronta.

Acurácia da Floresta: 96.39%

	precision	recall	f1-score	support
0	1.00	1.00	1.00	33
1	0.90	0.96	0.93	28
2	0.97	0.97	0.97	33
3	0.97	0.94	0.96	34
4	1.00	0.98	0.99	46
5	0.94	0.98	0.96	47
6	1.00	1.00	1.00	35
7	0.94	1.00	0.97	34
8	1.00	0.93	0.97	30
9	0.92	0.88	0.90	40
accuracy			0.96	360
macro avg	0.96	0.96	0.96	360
weighted avg	0.96	0.96	0.96	360

Mostrando um resultado agradável de mais de 96% ao colocar mais árvores para discutir qual número seria o daquele exemplo.

É notável que ao aumentar de 20 para 100 árvores tivemos um aumento de precisão, mas não é verdade que aumentando ainda mais teríamos um resultado mais preciso.

> Árvore 498 pronta.

> Árvore 499 pronta.

> Árvore 500 pronta.

Acurácia da Floresta: 96.39%

	precision	recall	f1-score	support
0	1.00	1.00	1.00	33
1	0.88	1.00	0.93	28
2	1.00	0.97	0.98	33
3	0.97	0.94	0.96	34
4	1.00	0.98	0.99	46
5	0.94	0.98	0.96	47
6	1.00	1.00	1.00	35
7	0.97	0.97	0.97	34
8	1.00	0.93	0.97	30
9	0.90	0.88	0.89	40
accuracy			0.96	360
macro avg	0.97	0.96	0.96	360
weighted avg	0.97	0.96	0.96	360

precision : taxa de acerto do número

recall : taxa de acerto da quantidade de números no teste

f1-score : média harmônica entre a Precisão e o Recall

support : quantidade de vezes que o número apareceu

Implementação da aplicação web

Agora, mostramos como implementamos a aplicação web para entendermos o que a IA estava “enxergando” ao realizar os testes.

Funções:

- `treinar_modelo_completo()`
- `calcular_importancia_pixel(floresta)`

```
@st.cache_resource
def treinar_modelo_completo():
    digits = load_digits()

    # Binarização: Transforma tons de cinza em 0 (preto) ou 1 (branco)
    # Isso facilita a decisão do algoritmo ID3 (baseado em Sim/Não)
    X_bin = (digits.data > 8).astype(int)
    y = digits.target

    examples = []
    for i in range(len(X_bin)):
        # Cria um dicionário para cada imagem onde cada pixel é um atributo (px_0, px_1...)
        ex = {f"px_{j}": X_bin[i][j] for j in range(64)}
        # Adiciona características extras (formas) extraídas da sua função personalizada
        ex.update(extrair_todas_as_formas(X_bin[i]))
        ex['target'] = y[i]
        examples.append(ex)

    # Define a lista de nomes de todos os atributos que a IA pode usar
    attrs = [f"px_{i}" for i in range(64)] + ['f_reta_topo', 'f_curva_base', 'f_muitos_cruzamentos', 'f_centro_preenchido']

    # Separa 80% para treinar a IA e 20% para testar se ela aprendeu mesmo
    train, test = train_test_split(examples, test_size=0.2, random_state=42)

    # Chama a sua função de treino para criar a "Floresta" (conjunto de árvores ID3)
    floresta = treinar_random_forest(train, attrs, n_arvores=100, profundidade_maxima=12)

    return floresta, test
```

```

def calcular_importancia_pixel(floresta):
    """
    Esta função percorre todas as árvores da floresta e conta
    quantas vezes cada pixel (atributo) foi usado para tomar uma decisão.
    """
    contagem = Counter()
    for arvore in floresta:
        def percorrer(no):
            if isinstance(no, dict): # Se o nó não é uma folha (é uma decisão)
                atrib = list(no.keys())[0]
                contagem[atrib] += 1 # Conta o atributo usado
                # Continua descendo na árvore recursivamente
                for sub in no[atrib].values(): percorrer(sub)
            percorrer(arvore)

    # Organiza a contagem em uma matriz 8x8 (formato da imagem)
    imp_matrix = np.zeros(64)
    for i in range(64):
        imp_matrix[i] = contagem.get(f"px_{i}", 0)
    return imp_matrix.reshape(8, 8)

# --- INTERFACE VISUAL (Colunas) ---
col1, col2 = st.columns([1, 1])

with col1:
    st.subheader("Visualização dos Pixels")
    fig, ax = plt.subplots()

    # Desenha a imagem do dígito selecionado
    img_data = np.array([exemplo_selecionado[f"px_{i}"] for i in range(64)]).reshape(8, 8)
    ax.imshow(img_data, cmap='gray_r')

    # Sobre põe um "mapa de calor" vermelho mostrando onde a IA mais "olha"
    importancia = calcular_importancia_pixel(floresta)
    ax.imshow(importancia, cmap='Reds', alpha=0.4) # alpha 0.4 deixa transparente
    ax.axis('off')
    st.pyplot(fig)
    st.info("O tom vermelho indica os pixels mais consultados pela floresta.")

```

```

with col2:
    st.subheader("Resultado da Votação")

    # Faz cada árvore da floresta dar o seu "palpite" individual
    votos = [prever_individual(arv, exemplo_selecionado) for arv in floresta]
    votos = [v for v in votos if v is not None] # Remove votos nulos

    # Contabiliza qual número recebeu mais votos
    contagem_votos = Counter(votos)
    previsao = contagem_votos.most_common(1)[0][0] if votos else 0
    real = exemplo_selecionado['target']
    confianca = (contagem_votos[previsao] / len(votos)) * 100 if votos else 0

    # Mostra o resultado principal com uma métrica visual
    st.metric("Predição da Floresta", f"Dígito {previsao}", delta=f"{confianca:.1f}% de votos")

    # Feedback de Acerto ou Erro
    if previsao == real:
        st.success(f"Acerto! O número real é {real}")
    else:
        st.error(f"Erro! O número real é {real}")

    # Gráfico de barras mostrando a distribuição dos votos para cada número (0-9)
    df_votos = pd.DataFrame({
        "Dígito": list(range(10)),
        "Votos": [contagem_votos.get(i, 0) for i in range(10)]
    })
    st.bar_chart(df_votos.set_index("Dígito"))

    # Mostra as características extras (como 'f_curva_base') que foram extraídas
    with st.expander("Ver Atributos de Forma extraídos"):
        formas = {k: exemplo_selecionado[k] for k in exemplo_selecionado if k.startswith('f_')}
        st.json(formas)

```

Funcionalidades:

treinar_modelo_completo() (Pipeline com Cache)

Esta função é o motor de inicialização do aplicativo. Ela prepara os dados e treina a inteligência artificial.

- O que faz: Realiza o carregamento do dataset, a binarização das imagens (preto e branco), a extração das formas geométricas e o treinamento da Random Forest com N árvores.
- O diferencial (@st.cache_resource): Este é um comando do Streamlit que salva o modelo treinado na memória (cache). Sem isso, a IA seria treinada do zero toda vez

que você movesse o slider, o que tornaria o uso muito lento. Com o cache, o treino ocorre apenas uma vez.

calcular_importancia_pixel(floresta) (Interpretabilidade)

Esta é uma das funções mais importantes para o relatório, pois ela permite "abrir a caixa preta" da IA e entender o que ela está pensando.

- O que faz: Ela percorre recursivamente cada nó de cada uma das N árvores da floresta e conta quantas vezes cada pixel (px_0 a px_63) foi utilizado como critério de decisão.
- Resultado: Ela gera uma matriz 8x8 onde os valores mais altos representam os pixels "mais valiosos" para a identificação dos números. No aplicativo, isso é transformado no Mapa de Calor (os tons de vermelho sobre a imagem).

Diferente dos outros scripts, o app.py orchestra elementos visuais para tornar o modelo interativo:

- Slider de Seleção: Permite navegar entre as centenas de imagens de teste para validar o desempenho da IA em tempo real.
- Colunas de Diagnóstico:

- Coluna 1 (Visualização): Utiliza a biblioteca matplotlib para sobrepor o dígito real com o mapa de calor de importância. Isso prova visualmente quais áreas do número a IA "olhou" para decidir se era um 8 ou um 0, por exemplo.
- Coluna 2 (Votação e Confiança): Realiza a inferência em tempo real. Ela coleta os votos de todas as árvores e calcula a confiança (porcentagem de consenso). Se 90 das 100 árvores votarem no número "3", o app exibe 90% de confiança.
- Gráfico de Barras (st.bar_chart): Exibe a distribuição exata dos votos. É útil para ver quando a IA ficou "em dúvida" (ex: quando os votos ficam divididos entre o número 0 e o 8).
- Expander de Atributos: Mostra os dados brutos das "Formas Extraídas" (f_reta_topo, etc.), permitindo conferir se a lógica de engenharia de atributos funcionou para aquele exemplo específico.